Compiled Solutions – Intermediate Division

1. **01-02 C2 Bit String Flicking**

   (RSHIFT–2 (RCIRC–8 (RSHIFT–2 10011))) = 00100
   (RSHIFT–2 (RCIRC–8 00100)) = (RSHIFT–2 10000) = 00100

2. **01-02 C2 Bit String Flicking**

   (RSHIFT–1 abcde) OR 10110 AND 00101 = 00101 0*01*
   0abcd OR (10110 AND 00101) = 0abcd OR 00100
   so a OR 0 = a and a OR 1 = 1 and 0a1cd = 00101.
   Therefore, a=0, c=0, and d=1 which gives the string 0*01*.

3. **02-03 C2 Bit String Flicking**

   LCIRC–2 (NOT 10110) OR (RSHIFT–2 (LCIRC–1 10100 AND NOT 00001)) 00111
   = LCIRC–2 01001 OR (RSHIFT–2 (LCIRC–1 10100 AND 11110)))
   = 00101 OR (RSHIFT–2 (01001 AND 11110))
   = 00101 OR (RSHIFT–2 01000)) = (00101 OR 00010) = 00111

4. **02-03 C2 Bit String Flicking**

   A27 XOR 4E2 = $EC5_{16}$
   1010 0010 0111 XOR 0100 1110 0010 =
   1110 1100 0101 = E C 5

5. **03-04 C2 Bit String Flicking**

   (NOT (RSHIFT-1 (RCIRC-2 (NOT (LSHIFT-1 01101))))) 11011
   (LSHIFT-1 01101) = 11010 (NOT 11010) = 00101
   (RCIRC-2 00101) = 01001 (RSHIFT-1 01001) = 00100
   (NOT 00100) = 11011

6. **03-04 C2 Bit String Flicking**

   Let X = ABCDE. Then (LCIRC-2 X) = CDEBA and 8
   (RCIRC-3 10101) = 10110
   10110 OR CDEBA = 10110
   1D11A = 10110 so all of the numbers match.
   Therefore, C = *, D = 0, E = *, A = 0, B = *
   ABCDE = 0**0* There are $2^3 = 8$ different values for ABCDE.

7. **04-05 C2 Bit String Flicking**

(RCIRC-3 (LSHIFT-1 (LCIRC-2 (RSHIFT-1 10110)))) = 01011
(RCIRC-3 (LSHIFT-1 (LCIRC-2 01011)))
(RCIRC-3 (LSHIFT-1 01101)) = (RCIRC-3 11010) = 01011

8. **04-05 C2 Bit String Flicking**

(LCIRC-2 01110) AND X = (RSHIFT-1 10011) 01**1
11001 AND abcde = 01001 implies
a = 0, b = 1, c = *, d = *, e = 1

9. **05-06 C2 Bit String Flicking**

Concatenate the bits starting in location 2 with a length of 3 to the 0001111100
right-hand side of the original string puts 110 at the end of the string.
(RCIRC-3 (LSHIFT–4 (CAT–2 3 0110111)))
(RCIRC-3 (LSHIFT–4 0110111110))
(RCIRC-3 1111100000) = 0001111100

10. **05-06 C2 Bit String Flicking**

(RCIRC–2 (LSHIFT–1 (NOT 01101))) OR (RSHIFT-3 (LCIRC-2 11100)) 00011
00001 OR 00010 = 00011

11. **06-07 C2 Bit String Flicking**

(RSHIFT-2 (10110 AND 10010)) = 00100 01111
(LCIRC-3 (01101 OR 10001)) = 01111
(00100 OR 01111) = 01111

12. **06-07 C2 Bit String Flicking**

(LSHIFT-2 (RCIRC-1 (NOT X))) = 10000 *011*
Let X= a b c d e and NOT X = A B C D E
(RCIRC-1 ABCDE) = E A B C D
(LSHIFT-2 EABCD) = B C D 0 0
B C D 0 0 = 1 0 0 0 0 $\Rightarrow$ B=1, C=0 and D=0.
X= * 0 1 1 *

13. **07-08 C2 Bit String Flicking**

The order of precedence is NOT, AND, and then OR. 11101
(NOT 10111 OR LCIRC-2 01110 OR LCIRC-1 00110 AND RSHIFT-2 10110)
= (01000 OR 11001 OR 01100 AND 00101)
= (01000 OR 11001 OR 00100) = (11001 OR 00100) = 11101

14. **07-08 C2 Bit String Flicking**

(LSHIFT-2 01011) = 01100 1***0
(RCIRC-2 abcde) = deabc
(01100 AND deabc) = 00100 ⇒ a = 1, b = *, c = *, d = *, e = 0

15. **08-09 C2 Bit String Flicking**

The order of precedence is XOR and then OR. 11011
(RCIRC-3 10110) OR (NOT 01110) XOR (LSHIFT-1 01101)
= 11010 OR 10001 XOR 11010 = 11010 OR 01011 = 11011

16. **08-09 C2 Bit String Flicking**

RHS: (RSHIFT-1 (RCIRC-2 10010)) = (RSHIFT-1 10100) = 01010 *101*
LHS: (LSHIFT-1 10111) AND abcde = 01110 AND abcde = 0bcd0
LHS = RHS ⇒ 0bcd0 = 01010 ⇒ a = *, b = 1, c = 0, d = 1, e = *
Therefore: *101* is the solution.

17. **09-10 C2 Bit String Flicking**

(LCIRC-2 (RSHIFT-1 01100)) AND (LSHIFT-1 (RCIRC-3 10011)) = 11000
(LCIRC-2 00110) AND (LSHIFT-1 01110) = 11000 AND 11100 = 11000

18. **09-10 C2 Bit String Flicking**

(LCIRC-2(RSHIFT-1 abcde)) OR (LSHIFT-1(RCIRC-2 00011)) = 11001 1*10*
(LCIRC-2 0abcd) OR (LSHIFT-1 11000) = 11001
bcd0a OR 10000 = 11001
⇒ b = *, d = 0, a = 1, c = 1, e = *

19. **10-11 C2 Bit String Flicking**

(RCIRC-2 (LSHIFT-1 (10011 AND NOT 00110))) 10000
= (RCIRC-2 (LSHIFT-1 (10011 AND 11001)))
= (RCIRC-2 (LSHIFT-1 10001)) = (RCIRC-2 00010) = 10000

20. **10-11 C2 Bit String Flicking**

(LCIRC-3 (RSHIFT-2 01110)) = (LCIRC-3 00011) = 11000 011**
(RCIRC-2 X) = (RCIRC-2 abcde) = deabc
So (LCIRC-3 (RSHIFT-2 01110)) OR (RCIRC-2 X) = 11011
Becomes 11000 OR deabc = 11011
11abc = 11011
∴ d == *, e = *, a = 0, b = 1, c = 1

21. **11-12 C2 Bit String Flicking**

(LSHIFT-2 00100) OR (RCIRC-2 01101) AND (NOT 01100) 10011
= 10000 OR (01011 AND 10011)
= 10000 OR 00011
= 10011

22. **11-12 C2 Bit String Flicking**

(LCIRC-2 (NOT 11011)) OR (RSHIFT-1 01101) 10110
= (LCIRC-2 00100) OR 00110
= 10000 OR 00110
= 10110

23. **12-13 C2 Bit String Flicking**

NOT 10110 OR 11001 AND 01110 01001
= (NOT 10110) OR (11001 AND 01110)
= 01001 OR 01000
= 01001

24. **12-13 C2 Bit String Flicking**

(RSHIFT-2 01101) AND (LCIRC-1 01100) OR (NOT 11011) 00100
= 00011 AND 11000 OR 00100 = 00000 OR 00100 = 00100

25. **13-14 C2 Bit String Flicking**

(NOT (10010 OR NOT 10011 AND 11011)) 00101
= (NOT (10010 OR 01100 AND 11011))
= (NOT (10010 OR 01000) = (NOT 11010)
= 00101

26. **13-14 C2 Bit String Flicking**

((LSHIFT–2 01101) OR 01011 AND (RCIRC–3 11011)) 11111
= 10100 OR 01011 AND 01111 = 10100 OR 01011
= 11111

27. **14-15 C2 Bit String Flicking**

10101 OR (NOT 01001) AND (10110 OR 01110) 10111
= 10101 OR 10110 AND 11110
= 10101 OR 10110
= 10111

28. **14-15 C2 Bit String Flicking**

(LCIRC-2(RSHIFT-1 01101)) OR (NOT(RCIRC-2(LSHIFT-1 01010))) 11010
= (LCIRC-2 00110) OR (NOT(RCIRC-2 10100))
= 11000 OR (NOT 00101)
= 11000 OR 11010
= 11010

29. **15-16 C2 Bit String Flicking**

(LSHIFT-1 (RCIRC-2 (RSHIFT-1 (LCIRC-3 01101)))) 10010
= (LSHIFT-1 (RCIRC-2 (RSHIFT-1 01011)))
= (LSHIFT-1 (RCIRC-2 00101))
= (LSHIFT-1 01001)
= 10010

30. **15-16 C2 Bit String Flicking**

(LCIRC-2(RSHIFT-1 01101)) AND ((RCIRC-1 01100) OR (NOT 01101)) 10000
= (LCIRC-2 00110) AND (00110 OR 10010)
= 11000 AND 10110
= 10000